

RIGHT JOIN, FULL OUTER JOIN y CROSS JOIN

En la clase anterior vimos que INNER JOIN y LEFT JOIN son las herramientas que vas a usar en la gran mayoría de tus consultas. Fácilmente cubren el 90% de los casos. Pero SQL tiene tres joins más que existen por una razón, y esta clase es sobre esos tres: RIGHT JOIN, FULL OUTER JOIN y CROSS JOIN.

Muchos programadores los ven en la documentación y los saltan pensando que son casos raros o rebuscados. La realidad es que hay escenarios donde son exactamente lo que necesitas, y si no los conoces, terminas armando consultas innecesariamente complejas para llegar al mismo resultado.

Primero, el mapa completo

Antes de entrar en cada uno, vale la pena ver el panorama general de los joins, porque así es más fácil entender dónde encaja cada pieza.

Ya conoces el **INNER JOIN**: te devuelve solo lo que coincide en ambas tablas. Y conoces el **LEFT JOIN**: te devuelve todo lo de la tabla izquierda (la del FROM) y lo que coincida de la derecha; donde no hay coincidencia, NULL.

Ahora sumamos tres más. El **RIGHT JOIN** es el espejo del LEFT: te trae todo lo de la tabla derecha y lo que coincida de la izquierda. El **FULL OUTER JOIN** es como decir "tráeme todo de ambos lados, coincida o no". Y el **CROSS JOIN** es algo completamente distinto: no busca coincidencias, sino que combina cada fila de una tabla con cada fila de la otra.

Con eso claro, vamos uno por uno.

RIGHT JOIN:

El RIGHT JOIN hace exactamente lo contrario del LEFT JOIN. Fuerza a que aparezcan todas las filas de la tabla de la **derecha** (la que escribes después del JOIN), y de la izquierda solo trae lo que coincide. Donde no hay coincidencia en la izquierda, te pone NULL.

Veámoslo con un caso concreto de TiendaLatam. La pregunta de negocio es: *"¿Cuántas ventas completadas atendió cada empleado activo en diciembre de 2024, incluyendo los que no atendieron ninguna?"*. La clave está en ese "incluyendo

los que no atendieron ninguna", porque eso significa que necesitas que aparezcan todos los empleados, hayan vendido o no.

Si un analista escribe la consulta partiendo de la tabla **Pedidos** en el FROM y hace un INNER JOIN con **Empleados**, solo va a ver a los seis empleados que tuvieron ventas ese mes. Los demás desaparecen, igual que pasaba en la clase anterior con los clientes.

Ahora, como la tabla principal del FROM es **Pedidos** (la izquierda), y lo que necesitas forzar son todos los **Empleados** (que está a la derecha), la solución directa es un RIGHT JOIN. Así le dices a la base de datos: "tráeme forzosamente todos los empleados, y si tienen pedidos en diciembre, agrégalos; si no, déjalo en cero".

Dicho esto, hay algo que tienes que saber: en la práctica, el RIGHT JOIN se usa muy poco. ¿Por qué? Porque siempre puedes lograr lo mismo cambiando el orden de las tablas y usando un LEFT JOIN. Es decir, en vez de poner **Pedidos** en el FROM y hacer RIGHT JOIN con **Empleados**, puedes poner **Empleados** en el FROM y hacer LEFT JOIN con **Pedidos**. El resultado es idéntico.

La convención en la industria es poner la tabla principal a la izquierda y usar LEFT JOIN. Es más legible y es lo que la mayoría de los equipos esperan ver. Pero conocer el RIGHT JOIN es importante por dos razones: primero, para poder leer código de otros que sí lo usan; y segundo, porque hay casos donde cambiar el orden de las tablas no es tan simple — por ejemplo, cuando ya tienes una consulta larga con varios joins y agregar uno más desde la derecha es más limpio que reestructurar todo.

FULL OUTER JOIN:

El FULL OUTER JOIN es la combinación de LEFT y RIGHT. Devuelve todas las filas de ambas tablas: donde hay coincidencia, une los datos normalmente; donde no la hay, en cualquiera de los dos lados, te pone NULL.

¿Y eso para qué sirve? Principalmente para **auditoría de datos**, es decir, para encontrar inconsistencias o registros huérfanos en ambas direcciones.

Pensemos en un ejemplo. Supongamos que quieres comparar las ventas de TiendaLatam en 2023 contra las de 2024, agrupadas por país. Podrías hacer cada consulta por separado, pero lo que realmente necesitas es ver ambos años lado a

lado en una sola tabla: el país, lo que vendió en 2023 y lo que vendió en 2024. El detalle es que puede haber países que tuvieron ventas en 2023 pero no en 2024, o viceversa. Si usaras un INNER JOIN entre ambas consultas, esos países desaparecerían. Si usaras un LEFT, solo verías los de un lado.

Con FULL OUTER JOIN ves todo: países que vendieron en ambos años, países que solo vendieron en uno y países que solo vendieron en el otro. Cada hueco (cada NULL) te está diciendo algo sobre el negocio.

Y hay un uso todavía más potente. ¿Recuerdas el patrón de LEFT JOIN + WHERE IS NULL que vimos en la clase anterior para encontrar filas huérfanas? Con FULL OUTER JOIN puedes hacer lo mismo, pero **en ambas direcciones a la vez**. Por ejemplo: ¿hay países en la tabla **Países** que no tienen ningún cliente? ¿Y hay clientes cuyo país no existe en la tabla **Países**? Una sola consulta con FULL OUTER JOIN y un filtro **WHERE** te responde ambas preguntas al mismo tiempo. Eso es auditoría de integridad de datos en su forma más directa.

CROSS JOIN:

El CROSS JOIN es fundamentalmente distinto a todo lo que hemos visto. Los otros joins buscan coincidencias entre tablas basándose en alguna condición (el famoso ON). El CROSS JOIN no tiene condición: lo que hace es tomar cada fila de una tabla y combinarla con cada fila de la otra. Esto se llama **producto cartesiano**.

Si la tabla A tiene 12 filas y la tabla B tiene 8, el resultado tiene 96 filas (12×8). Cada elemento de A aparece combinado con cada elemento de B.

En la clase vimos un ejemplo sencillo: combinar todos los países de TiendaLatam con todas las categorías de productos. El resultado es una lista con todas las combinaciones posibles de país y categoría. Eso por sí solo quizás no parece muy útil, pero es la base para algo que sí lo es: **análisis de cobertura**.

La idea es esta. Si generas todas las combinaciones posibles de país y categoría con un CROSS JOIN, y luego cruzas ese resultado con las ventas reales usando LEFT JOINS, puedes identificar qué combinaciones nunca tuvieron una venta. Es decir, puedes responder: "¿En qué países no se ha vendido nunca la categoría Deportes?" o "¿Qué categorías no tienen presencia en Chile?". Eso es inteligencia de negocio real, construida directamente desde SQL sin necesidad de exportar nada a Excel ni escribir código en Python.

Eso sí, hay una precaución importante con el CROSS JOIN: el tamaño del resultado. Si combinas 12 países con 8 categorías, son 96 filas, completamente manejable. Pero si se te ocurre cruzar una tabla de 100,000 clientes con una de 10,000 productos, estás generando mil millones de filas, y eso puede reventar el servidor. Siempre haz la multiplicación mentalmente antes de ejecutar un CROSS JOIN.

Resumiendo: cuándo usar cada uno

Después de estas dos clases, ya tienes el panorama completo de los joins en SQL. La forma más fácil de recordar cuándo usar cada uno es pensar en qué pregunta estás respondiendo.

Si necesitas **solo lo que coincide** en ambas tablas, INNER JOIN. Si necesitas **todo de un lado** y lo que coincida del otro, LEFT JOIN (o RIGHT JOIN si la tabla que quieres forzar está a la derecha). Si necesitas **todo de ambos lados** para detectar lo que falta en cualquier dirección, FULL OUTER JOIN. Y si necesitas **todas las combinaciones posibles** entre dos conjuntos de datos, CROSS JOIN.

Conocer estos cinco joins no es un lujo ni un tema de trivia para entrevistas. Es lo que separa a alguien que escribe consultas que funcionan de alguien que escribe consultas que responden la pregunta correcta.

Desafío de cierre

Usa FULL OUTER JOIN para encontrar países que tienen pedidos registrados pero no tienen clientes asociados en TiendaLatam. Pregúntate: ¿qué significa ese resultado para la integridad de los datos? ¿Debería ser posible que exista un pedido en un país donde no hay clientes? Comparte tu consulta y tu interpretación en los comentarios de la clase.